Water Segmentation using Multispectral and Optical Data

> **Project Objective**
>
> Develop a robust deep learning solution for accurately segmenting water bodies using multispectral and optical satellite data. This solution is vital for monitoring water resources, flood management, and environmental conservation, where precise segmentation of water bodies can significantly impact decision-making.

1. Project Overview

1.1 Data Specifications

- **Input Data:** 12-channel multispectral satellite imagery (Harmonized Sentinel-2/Landsat)
- **Resolution:** 128×128 pixels per image patch
- **Ground Sampling Distance:** 30 meters
- **Output:** Binary water mask (0 = non-water, 1 = water)
- **Data Format:** Multi-dimensional arrays with preserved spatial integrity

1.2 Spectral Bands

Band Index	Band Name	Description	Water Detection Relevance
0	Coastal Aerosol	443nm wavelength	Atmospheric correction, coastal water detection
1	Blue	490nm wavelength	Water penetration, sediment detection
2	Green	560nm wavelength	Water quality assessment, NDWI calculation
3	Red	665nm wavelength	Vegetation contrast, true color visualization
4	NIR	842nm wavelength	Strong water absorption, primary water indicator
5	SWIR1	1610nm wavelength	Water/soil separation, moisture content
6	SWIR2	2190nm wavelength	Enhanced water contrast, cloud masking
7	QA Band	Quality Assurance	Data quality filtering
8	Merit DEM	Digital Elevation Model	Topographic context for water bodies

9	Copernicus DEM	High-resolution elevation	Improved topographic features
10	ESA World Cover	Land cover classification	Contextual land use information
11	Water Occurrence	Historical water probability	Prior knowledge for water detection

2. Project Structure

```
water-segmentation/
├── data/
│   ├── raw/                    # Original satellite imagery
│   ├── processed/              # Preprocessed and normalized data
│   └── splits/                 # Train/validation/test splits
├── src/
│   ├── __init__.py
│   ├── data_loader.py          # Data loading and preprocessing
│   ├── models/
│   │   ├── __init__.py
│   │   ├── unet.py             # U-Net architecture
│   │   └── losses.py           # Custom loss functions
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── visualization.py    # Plotting and visualization tools
│   │   ├── metrics.py          # Evaluation metrics
│   │   └── transforms.py       # Data augmentation
│   ├── training/
│   │   ├── __init__.py
│   │   ├── trainer.py          # Training pipeline
│   │   └── config.py           # Configuration settings
│   └── inference/
│       ├── __init__.py
│       └── predictor.py        # Model inference
├── notebooks/
│   ├── 01_data_exploration.ipynb
│   ├── 02_visualization.ipynb
│   └── 03_model_evaluation.ipynb
├── configs/
│   └── config.yaml             # Training configuration
├── requirements.txt
├── setup.py
└── README.md
```

3. Complete Implementation

3.1 Requirements and Setup

`requirements.txt`

```
torch>=1.9.0
torchvision>=0.10.0
numpy>=1.21.0
opencv-python>=4.5.0
matplotlib>=3.4.0
seaborn>=0.11.0
scikit-learn>=1.0.0
tqdm>=4.62.0
```

```
albumentations>=1.1.0
segmentation-models-pytorch>=0.2.0
rasterio>=1.2.0
geopandas>=0.10.0
pillow>=8.3.0
pyyaml>=5.4.0
tensorboard>=2.7.0
```

3.2 Data Loading and Preprocessing

src/data_loader.py

```
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader
import rasterio
from pathlib import Path
import cv2
from sklearn.preprocessing import StandardScaler
import albumentations as A
from albumentations.pytorch import ToTensorV2

class WaterSegmentationDataset(Dataset):
    """Dataset class for water segmentation with 12-channel multispectral data."""

    def __init__(self, data_dir, split='train', transform=None, normalize=True):
        """
        Args:
            data_dir (str): Path to data directory
            split (str): 'train', 'val', or 'test'
            transform (albumentations.Compose): Data augmentation pipeline
            normalize (bool): Whether to normalize the data
        """
        self.data_dir = Path(data_dir)
        self.split = split
        self.transform = transform
        self.normalize = normalize

        # Load file paths
                    self.image_paths   =   sorted(list((self.data_dir   /   split   /
'images').glob('*.tif')))
                     self.mask_paths   =   sorted(list((self.data_dir   /   split   /
'masks').glob('*.tif')))

        assert len(self.image_paths) == len(self.mask_paths), \
                   f"Mismatch  between  images  ({len(self.image_paths)})  and  masks
({len(self.mask_paths)})"

        # Initialize normalizer
        if self.normalize and split == 'train':
            self._compute_normalization_stats()

    def _compute_normalization_stats(self):
        """Compute normalization statistics from training data."""
        print("Computing normalization statistics...")
        all_pixels = []

        for img_path in self.image_paths[:100]:  # Sample subset for efficiency
            with rasterio.open(img_path) as src:
                image = src.read()  # Shape: (12, 128, 128)
                image = np.transpose(image, (1, 2, 0))  # Shape: (128, 128, 12)
                all_pixels.append(image.reshape(-1, 12))
```

```
        all_pixels = np.vstack(all_pixels)
        self.mean = np.mean(all_pixels, axis=0)
        self.std = np.std(all_pixels, axis=0)

        # Avoid division by zero
        self.std = np.where(self.std == 0, 1, self.std)

                print(f"Normalization  stats  computed:  mean={self.mean[:4]},  std=
{self.std[:4]}")

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        # Load multispectral image (12 channels)
        with rasterio.open(self.image_paths[idx]) as src:
            image = src.read()  # Shape: (12, 128, 128)
               image = np.transpose(image, (1, 2, 0)).astype(np.float32)  # Shape:
(128, 128, 12)

        # Load binary mask
        with rasterio.open(self.mask_paths[idx]) as src:
            mask = src.read(1).astype(np.float32)  # Shape: (128, 128)

        # Normalize image
        if self.normalize:
            if hasattr(self, 'mean') and hasattr(self, 'std'):
                image = (image - self.mean) / self.std
            else:
                # Per-image normalization if global stats not available
                    image = (image - np.mean(image, axis=(0, 1))) / (np.std(image,
axis=(0, 1)) + 1e-8)

        # Apply augmentations
        if self.transform:
            # Albumentations expects mask as 2D array
            augmented = self.transform(image=image, mask=mask)
            image = augmented['image']
            mask = augmented['mask']

        # Convert to torch tensors
        if not isinstance(image, torch.Tensor):
             image = torch.from_numpy(image.transpose(2, 0, 1))  # Shape: (12, 128,
128)
        if not isinstance(mask, torch.Tensor):
            mask = torch.from_numpy(mask).unsqueeze(0)  # Shape: (1, 128, 128)

        return image, mask

def get_transforms(split='train'):
    """Get data augmentation transforms."""
    if split == 'train':
        return A.Compose([
            A.HorizontalFlip(p=0.5),
            A.VerticalFlip(p=0.5),
            A.Rotate(limit=90, p=0.5),
            A.RandomBrightnessContrast(p=0.3),
            A.GaussNoise(var_limit=(10.0, 50.0), p=0.2),
        ])
    else:
        return A.Compose([])

def create_data_loaders(data_dir, batch_size=16, num_workers=4):
    """Create train, validation, and test data loaders."""
```

```
    # Get transforms
    train_transform = get_transforms('train')
    val_transform = get_transforms('val')

    # Create datasets
    train_dataset = WaterSegmentationDataset(data_dir, 'train', train_transform)
    val_dataset = WaterSegmentationDataset(data_dir, 'val', val_transform)
    test_dataset = WaterSegmentationDataset(data_dir, 'test', val_transform)

    # Create data loaders
    train_loader = DataLoader(
        train_dataset, batch_size=batch_size, shuffle=True,
        num_workers=num_workers, pin_memory=True
    )
    val_loader = DataLoader(
        val_dataset, batch_size=batch_size, shuffle=False,
        num_workers=num_workers, pin_memory=True
    )
    test_loader = DataLoader(
        test_dataset, batch_size=batch_size, shuffle=False,
        num_workers=num_workers, pin_memory=True
    )

    return train_loader, val_loader, test_loader
```

3.3 U-Net Model Architecture

src/models/unet.py

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class DoubleConv(nn.Module):
    """Double convolution block used in U-Net."""

    def __init__(self, in_channels, out_channels, mid_channels=None):
        super(DoubleConv, self).__init__()
        if not mid_channels:
            mid_channels = out_channels
        self.double_conv = nn.Sequential(
            nn.Conv2d(in_channels, mid_channels, kernel_size=3, padding=1),
            nn.BatchNorm2d(mid_channels),
            nn.ReLU(inplace=True),
            nn.Conv2d(mid_channels, out_channels, kernel_size=3, padding=1),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        )

    def forward(self, x):
        return self.double_conv(x)

class Down(nn.Module):
    """Downscaling with maxpool then double conv."""

    def __init__(self, in_channels, out_channels):
        super(Down, self).__init__()
        self.maxpool_conv = nn.Sequential(
            nn.MaxPool2d(2),
            DoubleConv(in_channels, out_channels)
        )
```

```
    def forward(self, x):
        return self.maxpool_conv(x)

class Up(nn.Module):
    """Upscaling then double conv."""

    def __init__(self, in_channels, out_channels, bilinear=True):
        super(Up, self).__init__()

        if bilinear:
                            self.up  =  nn.Upsample(scale_factor=2,  mode='bilinear',
align_corners=True)
            self.conv = DoubleConv(in_channels, out_channels, in_channels // 2)
        else:
                     self.up  =  nn.ConvTranspose2d(in_channels,  in_channels  //  2,
kernel_size=2, stride=2)
            self.conv = DoubleConv(in_channels, out_channels)

    def forward(self, x1, x2):
        x1 = self.up(x1)
        # Input is CHW
        diffY = x2.size()[2] - x1.size()[2]
        diffX = x2.size()[3] - x1.size()[3]

        x1 = F.pad(x1, [diffX // 2, diffX - diffX // 2,
                        diffY // 2, diffY - diffY // 2])

        x = torch.cat([x2, x1], dim=1)
        return self.conv(x)

class OutConv(nn.Module):
    """Output convolution."""

    def __init__(self, in_channels, out_channels):
        super(OutConv, self).__init__()
        self.conv = nn.Conv2d(in_channels, out_channels, kernel_size=1)

    def forward(self, x):
        return self.conv(x)

class UNet(nn.Module):
    """
    U-Net architecture for water segmentation with 12 input channels.

    Args:
        n_channels (int): Number of input channels (12 for multispectral data)
        n_classes (int): Number of output classes (1 for binary segmentation)
        bilinear (bool): Whether to use bilinear upsampling
    """

    def __init__(self, n_channels=12, n_classes=1, bilinear=True):
        super(UNet, self).__init__()
        self.n_channels = n_channels
        self.n_classes = n_classes
        self.bilinear = bilinear

        self.inc = DoubleConv(n_channels, 64)
        self.down1 = Down(64, 128)
        self.down2 = Down(128, 256)
        self.down3 = Down(256, 512)
        factor = 2 if bilinear else 1
        self.down4 = Down(512, 1024 // factor)
        self.up1 = Up(1024, 512 // factor, bilinear)
        self.up2 = Up(512, 256 // factor, bilinear)
        self.up3 = Up(256, 128 // factor, bilinear)
        self.up4 = Up(128, 64, bilinear)
```

```
        self.outc = OutConv(64, n_classes)

    def forward(self, x):
        x1 = self.inc(x)
        x2 = self.down1(x1)
        x3 = self.down2(x2)
        x4 = self.down3(x3)
        x5 = self.down4(x4)
        x = self.up1(x5, x4)
        x = self.up2(x, x3)
        x = self.up3(x, x2)
        x = self.up4(x, x1)
        logits = self.outc(x)
        return logits

class AttentionUNet(nn.Module):
    """
    U-Net with attention gates for improved water segmentation.
    """

    def __init__(self, n_channels=12, n_classes=1):
        super(AttentionUNet, self).__init__()

        # Encoder
        self.enc1 = DoubleConv(n_channels, 64)
        self.enc2 = DoubleConv(64, 128)
        self.enc3 = DoubleConv(128, 256)
        self.enc4 = DoubleConv(256, 512)

        # Bottleneck
        self.bottleneck = DoubleConv(512, 1024)

        # Decoder with attention
        self.dec4 = DoubleConv(1024 + 512, 512)
        self.dec3 = DoubleConv(512 + 256, 256)
        self.dec2 = DoubleConv(256 + 128, 128)
        self.dec1 = DoubleConv(128 + 64, 64)

        # Attention gates
        self.att4 = AttentionBlock(512, 1024, 256)
        self.att3 = AttentionBlock(256, 512, 128)
        self.att2 = AttentionBlock(128, 256, 64)
        self.att1 = AttentionBlock(64, 128, 32)

        # Output
        self.final = nn.Conv2d(64, n_classes, kernel_size=1)

        # Pooling and upsampling
        self.pool = nn.MaxPool2d(2)
        self.up = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)

    def forward(self, x):
        # Encoder
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool(e1))
        e3 = self.enc3(self.pool(e2))
        e4 = self.enc4(self.pool(e3))

        # Bottleneck
        b = self.bottleneck(self.pool(e4))

        # Decoder with attention
        d4 = self.up(b)
        e4_att = self.att4(e4, b)
        d4 = torch.cat([d4, e4_att], dim=1)
        d4 = self.dec4(d4)
```

```
        d3 = self.up(d4)
        e3_att = self.att3(e3, d4)
        d3 = torch.cat([d3, e3_att], dim=1)
        d3 = self.dec3(d3)

        d2 = self.up(d3)
        e2_att = self.att2(e2, d3)
        d2 = torch.cat([d2, e2_att], dim=1)
        d2 = self.dec2(d2)

        d1 = self.up(d2)
        e1_att = self.att1(e1, d2)
        d1 = torch.cat([d1, e1_att], dim=1)
        d1 = self.dec1(d1)

        return self.final(d1)

class AttentionBlock(nn.Module):
    """Attention block for focusing on relevant features."""

    def __init__(self, F_g, F_l, F_int):
        super(AttentionBlock, self).__init__()
        self.W_g = nn.Sequential(
            nn.Conv2d(F_g, F_int, kernel_size=1, stride=1, padding=0, bias=True),
            nn.BatchNorm2d(F_int)
        )

        self.W_x = nn.Sequential(
            nn.Conv2d(F_l, F_int, kernel_size=1, stride=1, padding=0, bias=True),
            nn.BatchNorm2d(F_int)
        )

        self.psi = nn.Sequential(
            nn.Conv2d(F_int, 1, kernel_size=1, stride=1, padding=0, bias=True),
            nn.BatchNorm2d(1),
            nn.Sigmoid()
        )

        self.relu = nn.ReLU(inplace=True)

    def forward(self, g, x):
        g1 = self.W_g(g)
        x1 = self.W_x(x)
        psi = self.relu(g1 + x1)
        psi = self.psi(psi)

        return x * psi
```

3.4 Loss Functions and Metrics

src/models/losses.py

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class DiceLoss(nn.Module):
    """Dice loss for segmentation tasks."""

    def __init__(self, smooth=1e-6):
        super(DiceLoss, self).__init__()
```

```
        self.smooth = smooth

    def forward(self, predictions, targets):
        predictions = torch.sigmoid(predictions)

        # Flatten tensors
        predictions = predictions.view(-1)
        targets = targets.view(-1)

        intersection = (predictions * targets).sum()
           dice_score = (2. * intersection + self.smooth) / (predictions.sum() +
targets.sum() + self.smooth)

        return 1 - dice_score

class FocalLoss(nn.Module):
    """Focal loss for addressing class imbalance."""

    def __init__(self, alpha=1, gamma=2, smooth=1e-6):
        super(FocalLoss, self).__init__()
        self.alpha = alpha
        self.gamma = gamma
        self.smooth = smooth

    def forward(self, predictions, targets):
        # Apply sigmoid to get probabilities
        predictions = torch.sigmoid(predictions)

        # Flatten tensors
        predictions = predictions.view(-1)
        targets = targets.view(-1)

        # Calculate focal loss
        bce_loss = F.binary_cross_entropy(predictions, targets, reduction='none')
        pt = torch.where(targets == 1, predictions, 1 - predictions)
        focal_loss = self.alpha * (1 - pt) ** self.gamma * bce_loss

        return focal_loss.mean()

class CombinedLoss(nn.Module):
    """Combination of BCE and Dice loss."""

    def __init__(self, bce_weight=0.5, dice_weight=0.5):
        super(CombinedLoss, self).__init__()
        self.bce_weight = bce_weight
        self.dice_weight = dice_weight
        self.bce_loss = nn.BCEWithLogitsLoss()
        self.dice_loss = DiceLoss()

    def forward(self, predictions, targets):
        bce = self.bce_loss(predictions, targets)
        dice = self.dice_loss(predictions, targets)
        return self.bce_weight * bce + self.dice_weight * dice
```

src/utils/metrics.py

```
import torch
import numpy as np
from sklearn.metrics import confusion_matrix, precision_recall_fscore_support

class SegmentationMetrics:
    """Calculate segmentation metrics for water detection."""
```

```
    def __init__(self, threshold=0.5):
        self.threshold = threshold
        self.reset()

    def reset(self):
        """Reset all metrics."""
        self.predictions = []
        self.targets = []

    def update(self, predictions, targets):
        """Update metrics with new batch."""
        # Apply sigmoid and threshold
        predictions = torch.sigmoid(predictions)
        predictions = (predictions > self.threshold).float()

        # Move to CPU and convert to numpy
        predictions = predictions.cpu().numpy().flatten()
        targets = targets.cpu().numpy().flatten()

        self.predictions.extend(predictions)
        self.targets.extend(targets)

    def compute(self):
        """Compute all metrics."""
        predictions = np.array(self.predictions)
        targets = np.array(self.targets)

        # Confusion matrix
             tn, fp, fn, tp = confusion_matrix(targets, predictions, labels=[0,
1]).ravel()

        # Basic metrics
        precision = tp / (tp + fp) if (tp + fp) > 0 else 0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0
         f1_score = 2 * (precision * recall) / (precision + recall) if (precision +
recall) > 0 else 0
        accuracy = (tp + tn) / (tp + tn + fp + fn)

        # IoU (Intersection over Union)
        iou = tp / (tp + fp + fn) if (tp + fp + fn) > 0 else 0

        # Dice coefficient
        dice = 2 * tp / (2 * tp + fp + fn) if (2 * tp + fp + fn) > 0 else 0

        return {
            'accuracy': accuracy,
            'precision': precision,
            'recall': recall,
            'f1_score': f1_score,
            'iou': iou,
            'dice': dice,
            'confusion_matrix': {'TP': tp, 'TN': tn, 'FP': fp, 'FN': fn}
        }

    def get_current_metrics(self):
        """Get current metrics without resetting."""
        return self.compute()

def calculate_water_indices(image):
    """
    Calculate water indices from multispectral bands.

    Args:
        image: Tensor of shape (batch_size, 12, height, width)

    Returns:
```

```
        Dictionary of water indices
    """
    # Assuming band order: [Coastal, Blue, Green, Red, NIR, SWIR1, SWIR2, ...]
    green = image[:, 2, :, :]  # Green band
    red = image[:, 3, :, :]    # Red band
    nir = image[:, 4, :, :]    # NIR band
    swir1 = image[:, 5, :, :]  # SWIR1 band

    # NDWI (Normalized Difference Water Index)
    ndwi = (green - nir) / (green + nir + 1e-8)

    # MNDWI (Modified NDWI)
    mndwi = (green - swir1) / (green + swir1 + 1e-8)

    # AWEIsh (Automated Water Extraction Index shadow)
    blue = image[:, 1, :, :]
    swir2 = image[:, 6, :, :]
    aweish = blue + 2.5 * green - 1.5 * (nir + swir1) - 0.25 * swir2

    return {
        'ndwi': ndwi,
        'mndwi': mndwi,
        'aweish': aweish
    }
```

3.5 Visualization Tools

src/utils/visualization.py

```
import matplotlib.pyplot as plt
import numpy as np
import torch
import seaborn as sns
from matplotlib.colors import ListedColormap

def visualize_bands(image, band_names=None, figsize=(15, 10)):
    """
    Visualize all 12 spectral bands.

    Args:
        image: numpy array of shape (12, H, W) or (H, W, 12)
        band_names: list of band names
        figsize: figure size
    """
    if band_names is None:
        band_names = [
            'Coastal Aerosol', 'Blue', 'Green', 'Red', 'NIR', 'SWIR1', 'SWIR2',
              'QA Band', 'Merit DEM', 'Copernicus DEM', 'ESA World Cover', 'Water
Occurrence'
        ]

    if len(image.shape) == 3 and image.shape[0] == 12:
        image = np.transpose(image, (1, 2, 0))

    fig, axes = plt.subplots(3, 4, figsize=figsize)
    axes = axes.ravel()

    for i in range(12):
        band_data = image[:, :, i]

        # Normalize for visualization
        vmin, vmax = np.percentile(band_data, [2, 98])
```

```
        im = axes[i].imshow(band_data, cmap='viridis', vmin=vmin, vmax=vmax)
        axes[i].set_title(f'{band_names[i]}')
        axes[i].set_xticks([])
        axes[i].set_yticks([])
        plt.colorbar(im, ax=axes[i], fraction=0.046, pad=0.04)

    plt.tight_layout()
    plt.show()

def visualize_rgb_composites(image, figsize=(15, 5)):
    """
    Visualize RGB composites (True Color, False Color, SWIR).

    Args:
        image: numpy array of shape (12, H, W) or (H, W, 12)
    """
    if len(image.shape) == 3 and image.shape[0] == 12:
        image = np.transpose(image, (1, 2, 0))

    fig, axes = plt.subplots(1, 3, figsize=figsize)

    # True Color (Red, Green, Blue)
    true_color = image[:, :, [3, 2, 1]]  # Red, Green, Blue
    true_color = normalize_for_display(true_color)
    axes[0].imshow(true_color)
    axes[0].set_title('True Color (RGB)')
    axes[0].set_xticks([])
    axes[0].set_yticks([])

    # False Color (NIR, Red, Green)
    false_color = image[:, :, [4, 3, 2]]  # NIR, Red, Green
    false_color = normalize_for_display(false_color)
    axes[1].imshow(false_color)
    axes[1].set_title('False Color (NIR-R-G)')
    axes[1].set_xticks([])
    axes[1].set_yticks([])

    # SWIR Composite (SWIR2, SWIR1, Red)
    swir_color = image[:, :, [6, 5, 3]]  # SWIR2, SWIR1, Red
    swir_color = normalize_for_display(swir_color)
    axes[2].imshow(swir_color)
    axes[2].set_title('SWIR Composite')
    axes[2].set_xticks([])
    axes[2].set_yticks([])

    plt.tight_layout()
    plt.show()

def normalize_for_display(image):
    """Normalize image for RGB display."""
    # Clip extreme values
    p2, p98 = np.percentile(image, (2, 98))
    image = np.clip(image, p2, p98)

    # Normalize to 0-1
    image = (image - image.min()) / (image.max() - image.min())

    return image

def visualize_water_indices(image, figsize=(15, 5)):
    """Visualize water indices."""
    # Convert to torch tensor if needed
    if isinstance(image, np.ndarray):
        if len(image.shape) == 3 and image.shape[-1] == 12:
            image = np.transpose(image, (2, 0, 1))
```

```
        image = torch.from_numpy(image).unsqueeze(0)

    from .metrics import calculate_water_indices
    indices = calculate_water_indices(image)

    fig, axes = plt.subplots(1, 3, figsize=figsize)

    # NDWI
    ndwi = indices['ndwi'][0].numpy()
    im1 = axes[0].imshow(ndwi, cmap='RdYlBu', vmin=-1, vmax=1)
    axes[0].set_title('NDWI')
    axes[0].set_xticks([])
    axes[0].set_yticks([])
    plt.colorbar(im1, ax=axes[0], fraction=0.046, pad=0.04)

    # MNDWI
    mndwi = indices['mndwi'][0].numpy()
    im2 = axes[1].imshow(mndwi, cmap='RdYlBu', vmin=-1, vmax=1)
    axes[1].set_title('MNDWI')
    axes[1].set_xticks([])
    axes[1].set_yticks([])
    plt.colorbar(im2, ax=axes[1], fraction=0.046, pad=0.04)

    # AWEIsh
    aweish = indices['aweish'][0].numpy()
    im3 = axes[2].imshow(aweish, cmap='RdYlBu')
    axes[2].set_title('AWEIsh')
    axes[2].set_xticks([])
    axes[2].set_yticks([])
    plt.colorbar(im3, ax=axes[2], fraction=0.046, pad=0.04)

    plt.tight_layout()
    plt.show()

def visualize_predictions(image, ground_truth, prediction, threshold=0.5, figsize=
(15, 5)):
    """
    Visualize model predictions against ground truth.

    Args:
        image: Input image (RGB composite for display)
        ground_truth: Ground truth mask
        prediction: Model prediction (before sigmoid)
        threshold: Threshold for binary prediction
    """
    # Apply sigmoid and threshold to prediction
          pred_sigmoid   =   torch.sigmoid(prediction)   if   isinstance(prediction,
torch.Tensor) else prediction
    pred_binary = (pred_sigmoid > threshold).astype(np.float32)

    fig, axes = plt.subplots(1, 4, figsize=figsize)

    # Input image (RGB composite)
    if len(image.shape) == 3 and image.shape[0] == 12:
         rgb_image = np.transpose(image[[3, 2, 1], :, :], (1, 2, 0))  # Red, Green,
Blue
        rgb_image = normalize_for_display(rgb_image)
    else:
        rgb_image = image

    axes[0].imshow(rgb_image)
    axes[0].set_title('Input Image (RGB)')
    axes[0].set_xticks([])
    axes[0].set_yticks([])

    # Ground truth
```

```
    axes[1].imshow(ground_truth.squeeze(), cmap='Blues', vmin=0, vmax=1)
    axes[1].set_title('Ground Truth')
    axes[1].set_xticks([])
    axes[1].set_yticks([])

    # Prediction probability
    axes[2].imshow(pred_sigmoid.squeeze(), cmap='Blues', vmin=0, vmax=1)
    axes[2].set_title('Prediction Probability')
    axes[2].set_xticks([])
    axes[2].set_yticks([])

    # Binary prediction
    axes[3].imshow(pred_binary.squeeze(), cmap='Blues', vmin=0, vmax=1)
    axes[3].set_title('Binary Prediction')
    axes[3].set_xticks([])
    axes[3].set_yticks([])

    plt.tight_layout()
    plt.show()

def plot_training_history(history, figsize=(15, 5)):
    """Plot training and validation metrics."""
    fig, axes = plt.subplots(1, 3, figsize=figsize)

    # Loss
    axes[0].plot(history['train_loss'], label='Train Loss')
    axes[0].plot(history['val_loss'], label='Validation Loss')
    axes[0].set_title('Loss')
    axes[0].set_xlabel('Epoch')
    axes[0].set_ylabel('Loss')
    axes[0].legend()
    axes[0].grid(True)

    # IoU
    axes[1].plot(history['train_iou'], label='Train IoU')
    axes[1].plot(history['val_iou'], label='Validation IoU')
    axes[1].set_title('IoU Score')
    axes[1].set_xlabel('Epoch')
    axes[1].set_ylabel('IoU')
    axes[1].legend()
    axes[1].grid(True)

    # F1 Score
    axes[2].plot(history['train_f1'], label='Train F1')
    axes[2].plot(history['val_f1'], label='Validation F1')
    axes[2].set_title('F1 Score')
    axes[2].set_xlabel('Epoch')
    axes[2].set_ylabel('F1 Score')
    axes[2].legend()
    axes[2].grid(True)

    plt.tight_layout()
    plt.show()

def plot_confusion_matrix(confusion_matrix, figsize=(8, 6)):
    """Plot confusion matrix."""
    labels = ['Non-Water', 'Water']
    cm_array = np.array([[confusion_matrix['TN'], confusion_matrix['FP']],
                         [confusion_matrix['FN'], confusion_matrix['TP']]])

    plt.figure(figsize=figsize)
    sns.heatmap(cm_array, annot=True, fmt='d', cmap='Blues',
                xticklabels=labels, yticklabels=labels)
    plt.title('Confusion Matrix')
    plt.xlabel('Predicted')
```

```
    plt.ylabel('Actual')
    plt.show()
```

3.6 Training Pipeline

src/training/trainer.py

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim.lr_scheduler import ReduceLROnPlateau
import numpy as np
from tqdm import tqdm
import logging
from pathlib import Path
import json

from ..models.unet import UNet
from ..models.losses import CombinedLoss, DiceLoss, FocalLoss
from ..utils.metrics import SegmentationMetrics

class WaterSegmentationTrainer:
    """Trainer class for water segmentation model."""

    def __init__(self, config):
        """
        Initialize trainer with configuration.

        Args:
            config: Training configuration dictionary
        """
        self.config = config
        self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

        # Initialize model
        self.model = self._build_model()
        self.model.to(self.device)

        # Initialize loss function
        self.criterion = self._build_loss_function()

        # Initialize optimizer
        self.optimizer = self._build_optimizer()

        # Initialize scheduler
        self.scheduler = ReduceLROnPlateau(
            self.optimizer, mode='min', patience=5, factor=0.5, verbose=True
        )

        # Initialize metrics
        self.train_metrics = SegmentationMetrics()
        self.val_metrics = SegmentationMetrics()

        # Training history
        self.history = {
            'train_loss': [], 'val_loss': [],
            'train_iou': [], 'val_iou': [],
            'train_f1': [], 'val_f1': [],
            'train_precision': [], 'val_precision': [],
            'train_recall': [], 'val_recall': []
        }
```

```
        # Setup logging
        self._setup_logging()

        # Best model tracking
        self.best_val_iou = 0.0
        self.best_model_path = Path(config['save_dir']) / 'best_model.pth'

    def _build_model(self):
        """Build the segmentation model."""
        model_config = self.config['model']

        if model_config['architecture'] == 'unet':
            return UNet(
                n_channels=model_config['n_channels'],
                n_classes=model_config['n_classes'],
                bilinear=model_config.get('bilinear', True)
            )
        else:
                                        raise   ValueError(f"Unknown   architecture:
{model_config['architecture']}")

    def _build_loss_function(self):
        """Build loss function."""
        loss_config = self.config['loss']

        if loss_config['type'] == 'combined':
            return CombinedLoss(
                bce_weight=loss_config.get('bce_weight', 0.5),
                dice_weight=loss_config.get('dice_weight', 0.5)
            )
        elif loss_config['type'] == 'dice':
            return DiceLoss()
        elif loss_config['type'] == 'focal':
            return FocalLoss(
                alpha=loss_config.get('alpha', 1),
                gamma=loss_config.get('gamma', 2)
            )
        elif loss_config['type'] == 'bce':
            return nn.BCEWithLogitsLoss()
        else:
            raise ValueError(f"Unknown loss type: {loss_config['type']}")

    def _build_optimizer(self):
        """Build optimizer."""
        opt_config = self.config['optimizer']

        if opt_config['type'] == 'adam':
            return optim.Adam(
                self.model.parameters(),
                lr=opt_config['lr'],
                weight_decay=opt_config.get('weight_decay', 1e-4)
            )
        elif opt_config['type'] == 'adamw':
            return optim.AdamW(
                self.model.parameters(),
                lr=opt_config['lr'],
                weight_decay=opt_config.get('weight_decay', 1e-4)
            )
        else:
            raise ValueError(f"Unknown optimizer type: {opt_config['type']}")

    def _setup_logging(self):
        """Setup logging."""
        log_dir = Path(self.config['save_dir'])
        log_dir.mkdir(parents=True, exist_ok=True)
```

```
        logging.basicConfig(
            level=logging.INFO,
            format='%(asctime)s - %(levelname)s - %(message)s',
            handlers=[
                logging.FileHandler(log_dir / 'training.log'),
                logging.StreamHandler()
            ]
        )
        self.logger = logging.getLogger(__name__)

    def train_epoch(self, train_loader):
        """Train for one epoch."""
        self.model.train()
        self.train_metrics.reset()

        total_loss = 0.0
        num_batches = len(train_loader)

        progress_bar = tqdm(train_loader, desc='Training')

        for batch_idx, (images, masks) in enumerate(progress_bar):
            images = images.to(self.device)
            masks = masks.to(self.device)

            # Forward pass
            self.optimizer.zero_grad()
            outputs = self.model(images)
            loss = self.criterion(outputs, masks)

            # Backward pass
            loss.backward()
            self.optimizer.step()

            # Update metrics
            total_loss += loss.item()
            self.train_metrics.update(outputs, masks)

            # Update progress bar
            progress_bar.set_postfix({
                'loss': f'{loss.item():.4f}',
                'avg_loss': f'{total_loss / (batch_idx + 1):.4f}'
            })

        # Calculate epoch metrics
        epoch_metrics = self.train_metrics.compute()
        avg_loss = total_loss / num_batches

        return avg_loss, epoch_metrics

    def validate_epoch(self, val_loader):
        """Validate for one epoch."""
        self.model.eval()
        self.val_metrics.reset()

        total_loss = 0.0
        num_batches = len(val_loader)

        with torch.no_grad():
            progress_bar = tqdm(val_loader, desc='Validation')

            for batch_idx, (images, masks) in enumerate(progress_bar):
                images = images.to(self.device)
                masks = masks.to(self.device)

                # Forward pass
                outputs = self.model(images)
```

```
                loss = self.criterion(outputs, masks)

                # Update metrics
                total_loss += loss.item()
                self.val_metrics.update(outputs, masks)

                # Update progress bar
                progress_bar.set_postfix({
                    'loss': f'{loss.item():.4f}',
                    'avg_loss': f'{total_loss / (batch_idx + 1):.4f}'
                })

        # Calculate epoch metrics
        epoch_metrics = self.val_metrics.compute()
        avg_loss = total_loss / num_batches

        return avg_loss, epoch_metrics

    def train(self, train_loader, val_loader):
        """Main training loop."""
        num_epochs = self.config['num_epochs']

        self.logger.info(f"Starting training for {num_epochs} epochs")
        self.logger.info(f"Device: {self.device}")
                 self.logger.info(f"Model  parameters:  {sum(p.numel()  for  p  in
self.model.parameters()):,}")

        for epoch in range(num_epochs):
            self.logger.info(f"\nEpoch {epoch + 1}/{num_epochs}")

            # Training phase
            train_loss, train_metrics = self.train_epoch(train_loader)

            # Validation phase
            val_loss, val_metrics = self.validate_epoch(val_loader)

            # Update learning rate
            self.scheduler.step(val_loss)

            # Update history
            self.history['train_loss'].append(train_loss)
            self.history['val_loss'].append(val_loss)
            self.history['train_iou'].append(train_metrics['iou'])
            self.history['val_iou'].append(val_metrics['iou'])
            self.history['train_f1'].append(train_metrics['f1_score'])
            self.history['val_f1'].append(val_metrics['f1_score'])
            self.history['train_precision'].append(train_metrics['precision'])
            self.history['val_precision'].append(val_metrics['precision'])
            self.history['train_recall'].append(train_metrics['recall'])
            self.history['val_recall'].append(val_metrics['recall'])

            # Log metrics
                     self.logger.info(f"Train  Loss:  {train_loss:.4f},  Val  Loss:
{val_loss:.4f}")
                self.logger.info(f"Train IoU: {train_metrics['iou']:.4f}, Val IoU:
{val_metrics['iou']:.4f}")
              self.logger.info(f"Train F1: {train_metrics['f1_score']:.4f}, Val F1:
{val_metrics['f1_score']:.4f}")

            # Save best model
            if val_metrics['iou'] > self.best_val_iou:
                self.best_val_iou = val_metrics['iou']
                self.save_model(self.best_model_path, epoch, val_metrics)
                            self.logger.info(f"New  best  model  saved  with  IoU:
{self.best_val_iou:.4f}")
```

```
            # Save checkpoint every 10 epochs
            if (epoch + 1) % 10 == 0:
                                checkpoint_path  =  Path(self.config['save_dir'])  /
f'checkpoint_epoch_{epoch + 1}.pth'
                self.save_model(checkpoint_path, epoch, val_metrics)

        self.logger.info("Training completed!")
        self.logger.info(f"Best validation IoU: {self.best_val_iou:.4f}")

    def save_model(self, path, epoch, metrics):
        """Save model checkpoint."""
        checkpoint = {
            'epoch': epoch,
            'model_state_dict': self.model.state_dict(),
            'optimizer_state_dict': self.optimizer.state_dict(),
            'scheduler_state_dict': self.scheduler.state_dict(),
            'metrics': metrics,
            'config': self.config,
            'history': self.history
        }
        torch.save(checkpoint, path)

    def load_model(self, path):
        """Load model checkpoint."""
        checkpoint = torch.load(path, map_location=self.device)
        self.model.load_state_dict(checkpoint['model_state_dict'])
        self.optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
        self.scheduler.load_state_dict(checkpoint['scheduler_state_dict'])
        self.history = checkpoint.get('history', self.history)
        return checkpoint['epoch'], checkpoint['metrics']
```

3.7 Configuration and Main Training Script

configs/config.yaml

```
# Water Segmentation Training Configuration

# Data configuration
data:
  data_dir: "./data"
  batch_size: 16
  num_workers: 4
  pin_memory: true

# Model configuration
model:
  architecture: "unet"  # unet, attention_unet
  n_channels: 12
  n_classes: 1
  bilinear: true

# Loss configuration
loss:
  type: "combined"  # combined, dice, focal, bce
  bce_weight: 0.5
  dice_weight: 0.5
  alpha: 1.0  # for focal loss
  gamma: 2.0  # for focal loss

# Optimizer configuration
optimizer:
  type: "adamw"  # adam, adamw
```

```
  lr: 0.001
  weight_decay: 0.0001

# Training configuration
num_epochs: 100
save_dir: "./checkpoints"
seed: 42

# Evaluation configuration
eval:
  threshold: 0.5
  metrics: ["iou", "dice", "f1", "precision", "recall"]
```

train.py

```
#!/usr/bin/env python3
"""
Main training script for water segmentation model.
"""

import argparse
import yaml
import torch
import numpy as np
import random
from pathlib import Path

from src.data_loader import create_data_loaders
from src.training.trainer import WaterSegmentationTrainer
from src.utils.visualization import plot_training_history

def set_seed(seed):
    """Set random seed for reproducibility."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

def load_config(config_path):
    """Load configuration from YAML file."""
    with open(config_path, 'r') as f:
        config = yaml.safe_load(f)
    return config

def main():
    parser = argparse.ArgumentParser(description='Train water segmentation model')
    parser.add_argument('--config', type=str, default='configs/config.yaml',
                        help='Path to configuration file')
    parser.add_argument('--data_dir', type=str, default=None,
                        help='Path to data directory (overrides config)')
    parser.add_argument('--resume', type=str, default=None,
                        help='Path to checkpoint to resume training')

    args = parser.parse_args()

    # Load configuration
    config = load_config(args.config)

    # Override data directory if provided
    if args.data_dir:
        config['data']['data_dir'] = args.data_dir
```

```
    # Set random seed
    set_seed(config['seed'])

    # Create save directory
    save_dir = Path(config['save_dir'])
    save_dir.mkdir(parents=True, exist_ok=True)

    # Save configuration
    with open(save_dir / 'config.yaml', 'w') as f:
        yaml.dump(config, f, default_flow_style=False)

    print("=== Water Segmentation Training ===")
    print(f"Config: {args.config}")
    print(f"Data directory: {config['data']['data_dir']}")
    print(f"Save directory: {config['save_dir']}")
    print(f"Device: {'CUDA' if torch.cuda.is_available() else 'CPU'}")

    # Create data loaders
    print("\nCreating data loaders...")
    train_loader, val_loader, test_loader = create_data_loaders(
        data_dir=config['data']['data_dir'],
        batch_size=config['data']['batch_size'],
        num_workers=config['data']['num_workers']
    )

    print(f"Train samples: {len(train_loader.dataset)}")
    print(f"Validation samples: {len(val_loader.dataset)}")
    print(f"Test samples: {len(test_loader.dataset)}")

    # Initialize trainer
    print("\nInitializing trainer...")
    trainer = WaterSegmentationTrainer(config)

    # Resume training if checkpoint provided
    start_epoch = 0
    if args.resume:
        print(f"Resuming training from {args.resume}")
        start_epoch, _ = trainer.load_model(args.resume)
        start_epoch += 1

    # Start training
    print(f"\nStarting training from epoch {start_epoch}...")
    trainer.train(train_loader, val_loader)

    # Plot training history
    print("\nPlotting training history...")
    plot_training_history(trainer.history)

    # Evaluate on test set
    print("\nEvaluating on test set...")
    trainer.load_model(trainer.best_model_path)
    _, test_metrics = trainer.validate_epoch(test_loader)

    print("=== Test Results ===")
    for metric, value in test_metrics.items():
        if metric != 'confusion_matrix':
            print(f"{metric.upper()}: {value:.4f}")

    print(f"\nTraining completed! Best model saved at: {trainer.best_model_path}")

if __name__ == "__main__":
    main()
```

3.8 Inference and Evaluation

src/inference/predictor.py

```
```